

auto_ml

이진 분류(0/1) 문제를 자동으로 학습·스코어링하는 사내 라이브러리. 폐쇄 환경 이관 / 운영을 전제로 모듈화·설정 주도(YAML)로 설계되었다.

구성

auto_ml/	
├── config.py	설정 dataclass + YAML 로더
├── pipeline.py	학습 파이프라인 (auto-ml-train)
├── preprocessing/	1) 결측 → 2) 이상치 → 2.5) skew 변환 → 3) 스케일링
├── feature_selection/	Stability Selection 변수 선택
├── models/	LGBM / XGBoost / CatBoost 래퍼 + Trainer
├── tuning/	Optuna 베이지안 하이퍼파라미터 최적화
├── reporting/	HTML + PDF 리포트 (동일 내용)
├── scoring/	배치 스코어링 (auto-ml-score)
└── utils/	io / logger / validation

5단계 파이프라인

1. **전처리** — 결측 → 이상치 → skew 변환 → 스케일링 순서 고정. 학습 시 통계량을 저장해 스코어링 시 동일 적용.
2. **변수 선택** (선택) — Stability Selection (Meinshausen & Bühlmann, 2010). 전처리 직후 적용하여 안정적으로 선택되는 변수만 모델에 전달한다. `feature_selection.enabled: false` (기본값) 이면 건너뛴다.
3. **모형 적합** — LGBM / XGBoost / CatBoost 3종에 대해 (a) Optuna TPE 로 하이퍼파라미터 베이지안 최적화 → (b) StratifiedKFold OOF 평가 → (c) 학습 전체로 최종 fit → (d) 테스트 데이터로 평가. `primary_metric` (기본 ROC-AUC) 기준으로 best 모델을 선정한다.
4. **보고서 산출** — HTML / PDF 동일 내용 (Jinja2 + WeasyPrint). 모델 비교표, 튜닝 결과, ROC / PR 곡선, feature importance, score 분포, confusion matrix 포함.
5. **주기 스코어링** — 단일 artifact (preprocessor + model + metadata) 파일 1개 + 설정 YAML 1개로 운영 가능. cron 등에서 `auto-ml-score` 호출.

입력 데이터

- 학습 / 평가 데이터를 **별도 Parquet 파일** 로 받는다 (`train_data_path` , `test_data_path`). 내부에서 임의 분할하지 않는다.
- 학습/스코어링에 사용할 컬럼은 **CSV 파일** 로 명시한다. YAML 에서 `features_csv` 에 경로를 지정하면 `load_config` 가 자동으로 읽어 `cfg.features` 를 채운다.

CSV 형식 (필수 컬럼: `name`, `type`, `used`):

```
name,type,used
age,continuous,true
gender,category,true
income,continuous,true
internal_score,continuous,false
```

- `type`
- `continuous` — 연속형 (int / float). 결측·이상치·skew 변환·스케일링 적용.
- `category` — 범주형 (문자열 / 코드). 모델 래퍼에서 자체 인코딩 처리.
- `used`
- `true` 인 행만 학습/스코어링에 사용된다 (`false` /공백 → 제외).
- 운영 중 컬럼을 켜고 끌 때 코드 변경 없이 본 CSV 만 수정하면 된다.

```
features_csv: ./configs/features.csv
```

이상치 처리 (백분위수 원저라이징)

전처리 2단계. 학습 데이터의 컬럼별 분위수 (`quantile(lower_q)`, `quantile(upper_q)`) 를 경계로 사용해 원저라이징하고, 스코어링 시 학습 시 저장한 동일 경계를 그대로 적용한다.

```
preprocessing:
  outlier_method: percentile      # percentile | none
  outlier_lower_quantile: 0.01
  outlier_upper_quantile: 0.99
  outlier_action: clip           # clip | null_then_impute
```

- `outlier_method`
- `percentile` — 학습 분포의 (`lower_q`, `upper_q`) 분위수를 경계로 사용. (기본)
- `none` — 비활성.
- `outlier_action`
- `clip` — 경계 밖 값을 경계로 잘라낸다. (기본)
- `null_then_impute` — 경계 밖을 NaN 으로 만든 뒤 학습 시 median 으로 다시 채운다.
- `outlier_lower_quantile < outlier_upper_quantile` 이고 둘 다 `[0, 1]` 범위여야 한다 (위반 시 `ValueError`).

Skew 변환

전처리 2.5단계. 학습 데이터의 컬럼별 skew 를 측정해 임계값 초과인 변수만 자동으로 선택해 분포를 대칭화한다. 학습 시 추정된 변환 파라미터를 저장해 스코어링에 동일 적용한다.

```
preprocessing:
  skew_method: signed_log1p      # signed_log1p | quantile_normal | none
  skew_threshold: 1.0           # |skew| > threshold 인 컬럼만 변환
```

- `skew_method`
- `signed_log1p` — $\text{sign}(x) * \log1p(|x|)$. 음수까지 단조 보존, 무상태. (기본)
- `quantile_normal` — sklearn `QuantileTransformer` 로 학습 분포의 분위수를 표준정규로 매핑. 이상치/skew 모두에 강건. 학습 시 분위수 테이블을 저장.
- `none` — 비활성.
- 변환 대상은 학습 시 한 번만 결정된다. `|skew| > skew_threshold` 인 컬럼만 선택되며, 미선택 컬럼은 스코어링에서도 그대로 통과한다 (분포 일관성).
- 부스팅 트리는 단조 변환에 본질적으로 강건하지만, 선형/신경망 확장이나 분포 가정을 쓰는 진단에는 효과가 있다.

변수 선택 (Stability Selection)

전처리 직후, 모델 학습 직전에 적용되는 선택적 단계이다. `feature_selection.enabled: true` 로 켜면 학습 데이터에서 stratified 부분표본을 반복 추출하여 변수별 선택 빈도를 산출하고, 임계값 이상으로 자주 선택된 변수만 최종 학습에 사용한다.

```
feature_selection:
  enabled: true
  base_estimator: lasso          # lasso (L1 logistic) | lgbm (gain top-K)
  n_subsamples: 200             # 부분표본 추출 횟수
  subsample_ratio: 0.5          # 각 부분표본 크기 비율
  threshold: 0.6                # 채택 임계값 (보통 0.6~0.8)
  random_state: 42
  min_selected: 1               # fallback 최소 개수
```

- `base_estimator`
- `lasso` — L1 로지스틱 회귀. non-zero coefficient 인 변수를 선택.
- `lgbm` — LightGBM gain 중요도 상위 `lgbm_top_k` 개를 선택.
- `min_selected` — threshold 이상 변수가 부족하면 frequency 상위 N 개로 fallback 하여 모델 입력이 비는 것을 방지한다.
- 선택 결과는 artifact 메타데이터에 저장되어 스코어링 시 동일 컬럼 셋이 적용된다.

하이퍼파라미터 최적화

각 모델 블록에 `fixed_params` (고정) 와 `search_space` (탐색 범위) 를 둔다. `tuning.enabled: true` 이고 모델별 `search_space` 가 비어있지 않으면 Optuna TPE 가 KFold OOF `primary_metric` 을 최대화하는 파라미터를 찾는다.

```
tuning:
  enabled: true
  n_trials: 30
  cv_folds: 3
  random_state: 42

models:
  lgbm:
    enabled: true
    fixed_params: { objective: binary, metric: auc, verbose: -1, n_estimators: 2000 }
    search_space:
      learning_rate: { type: float, low: 0.01, high: 0.3, log: true }
      num_leaves:    { type: int,   low: 15,   high: 255 }
      reg_lambda:    { type: float, low: 1.0e-8, high: 10.0, log: true }
```

`search_space` 항목 형식:

- `type: float` — `low`, `high`, `log` (선택, 기본 false)
- `type: int` — `low`, `high`, `log` (선택), `step` (선택, 기본 1)
- `type: categorical` — `choices: [...]`

설치

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -e .
```

WeasyPrint 는 시스템 폰트와 일부 라이브러리(libpango 등) 가 필요하다. 폐쇄망에서는 wheelhouse + 시스템 패키지를 함께 배포한다.

사용

```
# 1) 더미 데이터 생성 (검증용)
python examples/make_dummy_data.py

# 2) 학습 — 산출물:
#   artifacts/models/best.joblib
#   artifacts/reports/report.html, report.pdf
auto-ml-train --config configs/example.yaml

# 3) 스코어링 — 산출물:
#   artifacts/scores/scores.parquet (id_columns + score + prediction)
auto-ml-score --config configs/example.yaml
```

코드에서 직접 호출하는 예시는 `examples/run_train.py`, `examples/run_score.py` 참고.

타이타닉 end-to-end 예시

실제 공개 데이터셋(Titanic, 891 rows) 으로 전체 파이프라인을 검증할 수 있다.

```
# 데이터 준비 (GitHub 에서 1회 다운로드 → data/titanic/{train,test,score_input}.parquet)
python examples/titanic/prepare_data.py

# 학습 — 산출물: artifacts/titanic/{models,reports,logs}/...
auto-ml-train --config examples/titanic/config.yaml

# 스코어링 — 산출물: artifacts/titanic/scores/scores.parquet
auto-ml-score --config examples/titanic/config.yaml
```

검증 시 약 10초 내외 (Optuna 10 trials × 3 모델 × 3 fold) 에 best 모델 ROC-AUC ≈ 0.86 을 재현한다. 폐쇄망에서는 `TITANIC_CSV` 환경변수에 사전 배포한 CSV 경로를 지정하면 `prepare_data.py` 가 네트워크 없이 동일 산출물을 만든다.

폐쇄망 이관

다음 4가지만 옮기면 된다:

1. 패키지 wheel (`pip wheel -r requirements.txt -w wheelhouse/`)
2. 본 repo 자체 (또는 `pip install -e .` 으로 wheel 생성)
3. 학습 산출물 `artifacts/models/best.joblib`
4. 스코어링 설정 YAML

설정

전체 옵션은 `configs/example.yaml` 참고. 모든 키는 `auto_ml/config.py` 의 `dataclass` 와 1:1 대응한다.

작업 로그

실행마다 stage(`train` / `score`) 별 별도 로그 파일이 자동 생성된다.

```
artifacts/logs/
├── train_20260425_143052.log
└── score_20260425_180001.log
```

설정 (`configs/example.yaml` 의 `logging` 섹션):

```
logging:
  log_dir: ./artifacts/logs
  level: INFO          # DEBUG | INFO | WARNING | ERROR
  to_stdout: true
  to_file: true
```

콘솔 출력과 파일 출력은 동시에 활성화 가능하며, `to_file: false` 로 두면 파일은 만들지 않는다. 각 로그는 시작/종료 마커, 단계별 진행, 모델 튜닝 결과, 산출물 경로, 총 소요 시간을 포함한다.

운영 메모

- 타깃은 0/1 만 허용 (`utils/validation.py` 가 검증).
- best 선정은 테스트 데이터 점수 기준.
- 학습 시 사용한 features 정의가 artifact 메타데이터에 저장되어 스코어링 시 동일 컬럼 셋·동일 전처리·동일 모델로 처리된다.
- 스코어링 결과 컬럼: `<id_columns> + score + prediction`.
- 학습 데이터에서 어떤 수치형 컬럼이 전부 NaN 이면 결측·이상치 단계가 명시적 `ValueError` 로 실패한다 (silent NaN 전파 방지). `features.csv` 의 `used` 를 `false` 로 두거나 해당 단계를 비활성화/ `constant` 전략으로 전환.